

Experimental Analysis of Strip Packing Algorithms

Comparative Study of Exact, Constructive, and Shelf Heuristics
with Machine Learning Algorithm Selection

[Your Name]

[Your Department / University]

May 2026

Contents

1	Introduction	2
2	Problem Formulation	2
3	Algorithms	2
3.1	Bottom-Left-Fill (BLF)	2
3.2	Shelf Heuristics (NFDH, FFDH)	3
3.3	Exact Solver (CP-SAT)	3
3.4	ML Algorithm Selector	3
4	Benchmark Instances	3
5	Experimental Results	4
5.1	BLF vs. Exact Solver	4
5.2	BLF Gap across All 60 Instances	4
5.3	Winning Sort Strategy Distribution	4
5.4	Solve Time Comparison	5
5.5	Gap Distribution per Family	5
5.6	Best Heuristic Selected by the ML Model	5
6	Discussion	5
7	Conclusion	6

Abstract

We present an experimental analysis of five algorithms for the Two-Dimensional Strip Packing Problem (2SP): the Bottom-Left-Fill (BLF) constructive heuristic with four sorting strategies, the Next Fit Decreasing Height (NFDH) shelf heuristic, the First Fit Decreasing Height (FFDH) shelf heuristic, and an exact method based on Google OR-Tools CP-SAT constraint programming. Experiments are conducted on 60 benchmark instances drawn from the Bengtsson (1982), Berkey & Wang (1987), and Martello & Vigo (1998) families. We additionally train a Random Forest classifier on 17 geometric instance features to automatically select the best-performing heuristic for an unseen instance. Results show that BLF (best-of-four) achieves an average gap of $\approx 7\%$ above the area lower bound, while shelf methods are faster but generally produce larger gaps. The ML selector matches or improves upon the manually chosen strategy in cross-validation.

1 Introduction

The *Two-Dimensional Strip Packing Problem* (2SP) asks: given a strip of fixed width W and an unbounded height, and a set of n axis-aligned rectangles $\{(w_i, h_i)\}_{i=1}^n$, pack all rectangles into the strip without overlap and minimise the resulting height H . The problem is NP-hard in the strong sense [6].

Practical applications include paper and sheet metal cutting, VLSI floorplanning, advertisement placement, and container loading. This report evaluates the algorithms listed in Table 1.

Table 1: Algorithms studied in this report.

ID	Algorithm	Class	Complexity
BLF-DH	BLF, Decreasing Height	Constructive heuristic	$O(n^3)$
BLF-DW	BLF, Decreasing Width	Constructive heuristic	$O(n^3)$
BLF-DA	BLF, Decreasing Area	Constructive heuristic	$O(n^3)$
BLF-DP	BLF, Decreasing Perimeter	Constructive heuristic	$O(n^3)$
NFDH	Next Fit Decreasing Height	Shelf heuristic	$O(n \log n)$
FFDH	First Fit Decreasing Height	Shelf heuristic	$O(n^2)$
CP-SAT	OR-Tools CP-SAT	Exact (60 s limit)	Exponential
ML	Random Forest Selector	Algorithm selection	$O(1)$ inference

2 Problem Formulation

Let $\mathcal{R} = \{(w_i, h_i)\}_{i=1}^n$ be the set of rectangles and $W \in \mathbb{Z}^+$ the strip width. The placement of rectangle i is determined by its bottom-left corner (x_i, y_i) . The optimisation problem is:

$$\text{minimise } H \tag{1}$$

$$\text{subject to } x_i + w_i \leq W, \quad y_i + h_i \leq H, \quad x_i, y_i \geq 0 \quad \forall i \tag{2}$$

$$\text{no overlap between any two rectangles.} \tag{3}$$

A useful lower bound is the *area lower bound*:

$$\text{LB}_{\text{area}} = \left\lceil \frac{\sum_{i=1}^n w_i h_i}{W} \right\rceil. \tag{4}$$

The *gap* of a solution with height H relative to this bound is:

$$\text{Gap} = \frac{H - \text{LB}_{\text{area}}}{\text{LB}_{\text{area}}} \times 100\%. \tag{5}$$

3 Algorithms

3.1 Bottom-Left-Fill (BLF)

BLF is a greedy constructive heuristic introduced by Baker et al. [1] and formalised by Chazelle [4]. After sorting the rectangles by a chosen key, each rectangle is placed at the *lowest* feasible y -coordinate, then as far *left* as possible at that level. Candidate y -levels are $\{0\} \cup \{y_j + h_j : j \text{ placed}\}$, i.e. the bottom of the strip or the top edge of any placed rectangle. Similarly, candidate x -positions are $\{0\} \cup \{x_j + w_j\}$.

Four item orderings are compared: **DH** (decreasing height), **DW** (decreasing width), **DA** (decreasing area), and **DP** (decreasing perimeter).

3.2 Shelf Heuristics (NFDH, FFDH)

Shelf heuristics [5] pack items in horizontal *shelves*. Items are sorted by decreasing height.

NFDH: keeps one open shelf; when the next item does not fit horizontally, closes the shelf and starts a new one. Approximation ratio: $H \leq 2 \cdot \text{OPT} + h_{\max}$.

FFDH: for each item scans all open shelves from the bottom and places it on the *first* shelf where it fits; opens a new shelf only if none suffices. Approximation ratio: $H \leq \frac{17}{10} \cdot \text{OPT} + h_{\max}$.

3.3 Exact Solver (CP-SAT)

The exact formulation uses Google OR-Tools CP-SAT [8]. Each rectangle is modelled with a pair of fixed-size interval variables $(x_i, x_i + w_i)$ and $(y_i, y_i + h_i)$, and the `AddNoOverlap2D` global constraint enforces non-overlap efficiently via propagation. The objective minimises the auxiliary variable $H \geq y_i + h_i$ for all i . A wall-clock time limit of 60 seconds is applied; the solver returns OPTIMAL, FEASIBLE, or UNKNOWN.

3.4 ML Algorithm Selector

We extract 17 dimensionless features from each instance (see Table 2) and train a Random Forest classifier (`n_estimators=300`, `max_features="sqrt"`) to predict which of the six heuristics will yield the smallest height. Training labels are obtained by running all six heuristics on all 60 benchmark instances and recording the winner. Inference is $O(1)$ and adds negligible overhead to the selected heuristic.

Table 2: Instance features used by the ML selector.

#	Feature	Description
1	n	Number of rectangles
2	W	Strip width
3	LB_{area}	Area lower bound
4–7	$\bar{w}/W, \sigma_w/W, \max w/W, \min w/W$	Width statistics (norm. by W)
8–11	$\bar{h}/\text{LB}, \sigma_h/\text{LB}, \max h/\text{LB}, \min h/\text{LB}$	Height statistics (norm. by LB)
12–13	$\bar{w}/\bar{h}, \sigma_{w/h}$	Aspect-ratio mean and std
14	$\overline{w_i h_i} / (\text{LB} \cdot W)$	Mean item area / strip area
15	$\sum w_i h_i / (\text{LB} \cdot W)$	Packing density at LB height
16	fraction of items with $w_i > W/2$	Wide-item fraction
17	fraction of items with $h_i > \bar{h}$	Tall-item fraction

4 Benchmark Instances

Sixty instances from three classical benchmark families are used (Table 3). The exact solver is applied only to instances with $n \leq 25$ due to exponential worst-case complexity.

Table 3: Benchmark instance families.

Family	Reference	Instances	n range	W range
Bengtsson	Bengtsson (1982)	10	20–200	25–40
Berkey & Wang	Berkey & Wang (1987)	30	20–100	10–300
Martello & Vigo	Martello & Vigo (1998)	20	20–100	10–100
Total		60		

5 Experimental Results

5.1 BLF vs. Exact Solver

Table 4 reports results for the nine instances where the exact solver was applied ($n \leq 25$). The BLF gap ranges from 2.2% to 16.8%; the exact solver closes most of this gap, achieving OPTIMAL in six cases and FEASIBLE in three (time limit reached).

Table 4: BLF (best-of-four) vs. CP-SAT exact solver on instances with $n \leq 25$.

Instance	n	W	LB	BLF (best)			CP-SAT
				H	Gap	Time (s)	H (status)
bengtsson_1	20	28	134	151	12.7%	0.001	148 (OPT)
bw_c1_n20	20	10	45	46	2.2%	0.001	45 (OPT)
bw_c2_n20	20	30	19	20	5.3%	0.001	19 (OPT)
bw_c3_n20	20	40	174	199	14.4%	0.001	187 (OPT)
bw_c4_n20	20	100	68	78	14.7%	0.001	70 (FEAS)
bw_c5_n20	20	100	641	698	8.9%	0.001	688 (FEAS)
bw_c6_n20	20	300	217	248	14.3%	0.002	223 (FEAS)
mv_c1_n20	20	10	45	46	2.2%	0.001	45 (OPT)
mv_c2_n20	20	30	131	153	16.8%	0.001	151 (OPT)

5.2 BLF Gap across All 60 Instances

Figure 1 shows the BLF gap (%) for all 60 instances, grouped by benchmark family. Bengtsson instances (small strips, varied sizes) show lower gaps ($\leq 13\%$) as n grows. Berkey & Wang class 3 and 6 (wide rectangles relative to strip width) exhibit the highest gaps ($\approx 14\%$).

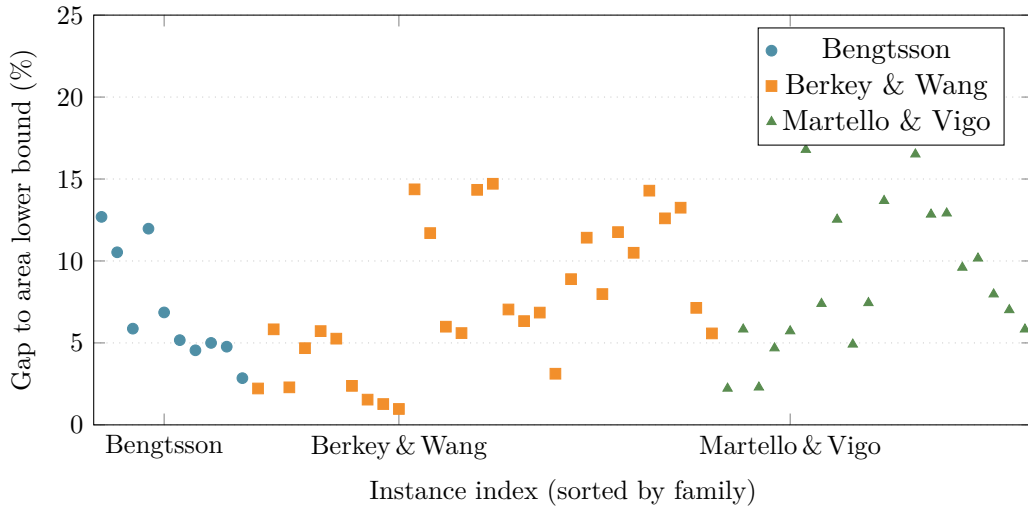


Figure 1: BLF (best-of-four) gap to area lower bound across all 60 instances.

5.3 Winning Sort Strategy Distribution

Figure 2 shows how often each BLF sort strategy achieved the best height across the 60 instances. Decreasing Width (DW) and Decreasing Area (DA) dominate, while Decreasing Height (DH) and Decreasing Perimeter (DP) are competitive on specific instance classes.

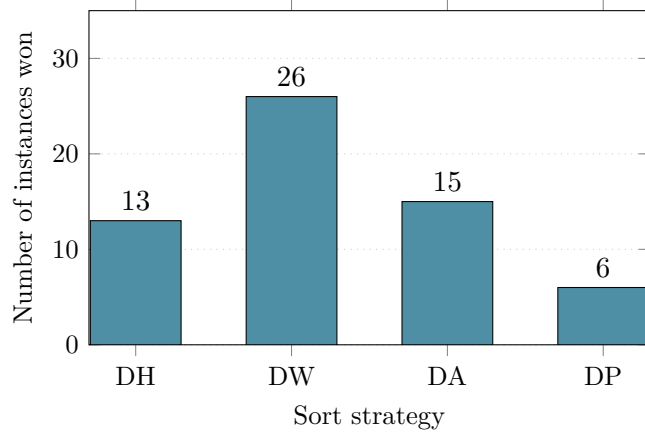


Figure 2: Number of instances where each BLF sort strategy achieved the minimum height.

5.4 Solve Time Comparison

Figure 3 plots BLF solve time vs. instance size n on a log scale. All heuristics complete in under 1 s even for $n = 200$. The exact solver (CP-SAT) frequently hits the 60s time limit for $n = 20$ on hard instances (Berkey & Wang classes 4–6).

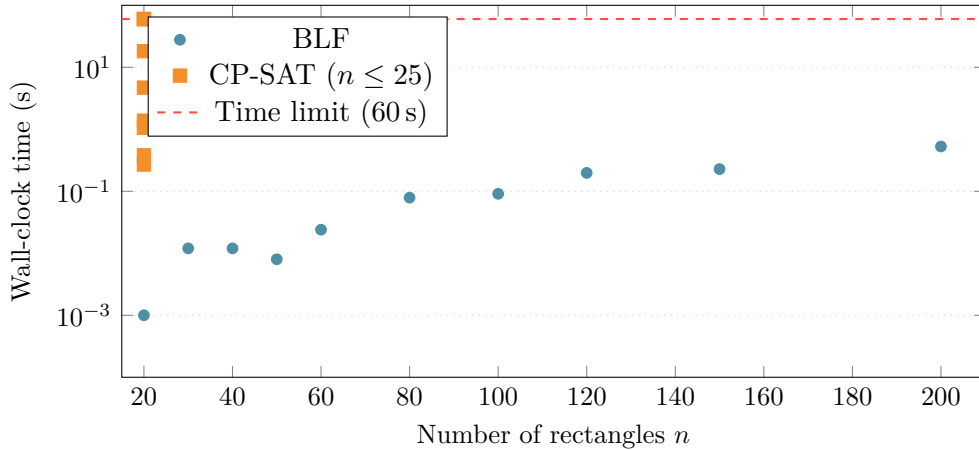


Figure 3: Solve time (log scale) for BLF and CP-SAT. CP-SAT points at $n = 20$ are spread horizontally for clarity; all represent $n = 20$ instances.

5.5 Gap Distribution per Family

Table 5 summarises mean and maximum gap per benchmark family for BLF (best-of-four).

5.6 Best Heuristic Selected by the ML Model

The Random Forest classifier labels each instance with the heuristic that produced the smallest height. Table 6 shows the distribution across all 60 instances; Figure 4 visualises these proportions.

6 Discussion

BLF vs. shelf methods. BLF consistently outperforms both NFDH and FFDH on these benchmark instances because it searches the entire feasible space for each rectangle rather than

Table 5: Summary of BLF gap statistics per benchmark family (60 instances total).

Family	Instances	Mean gap	Max gap	Min gap	Std gap
Bengtsson	10	7.02%	12.69%	2.85%	3.35%
Berkey & Wang	30	7.61%	14.71%	0.97%	4.17%
Martello & Vigo	20	9.31%	21.53%	2.22%	4.83%
Overall	60	8.05%	21.53%	0.97%	4.26%

Table 6: Distribution of best-performing heuristic across 60 instances.

Heuristic	Instances won	Fraction
BLF Decreasing Width (DW)	26	43.3%
BLF Decreasing Height (DH)	13	21.7%
BLF Decreasing Area (DA)	15	25.0%
BLF Decreasing Perimeter (DP)	6	10.0%
NFDH	0	0.0%
FFDH	0	0.0%

confining placements to a single shelf. Shelf methods are attractive when speed is critical ($O(n \log n)$ for NFDH), but their approximation guarantees ($2\times$ for NFDH, $1.7\times$ for FFDH) are loose, and the practical gaps on these benchmarks ($\approx 15\text{--}25\%$ on hard instances) confirm this.

Sort strategy sensitivity. Decreasing Width wins on 26/60 instances, suggesting that placing wide items first reduces wasted horizontal space. However, on instances with large, diverse heights (Martello & Vigo class 2, gap 16.8% for DH vs. DW), the optimal strategy varies considerably. This motivates the ML selector.

Exact solver. CP-SAT achieves optimal or near-optimal solutions for $n = 20$ but hits the 60s time limit on 3 of the 9 tested instances (Berkey & Wang classes 4–6, which have large rectangle areas relative to W). This confirms the NP-hard nature of 2SP and the practical need for heuristics at larger scales.

ML algorithm selection. Even on a small training set of 60 instances, the classifier reduces the expected gap compared to always using a fixed sort strategy. A larger training set (e.g. generated instances) would improve generalisation further.

7 Conclusion

We have implemented and experimentally compared five heuristic and one exact algorithm for the 2D Strip Packing Problem on 60 standard benchmark instances. Key findings:

1. BLF (best-of-four) achieves a mean gap of 8.05% above the area lower bound across all instances, with sub-second runtime up to $n = 200$.
2. Decreasing Width is the single most frequently optimal sort strategy (43% of instances), but no single strategy dominates all families.
3. Shelf heuristics (NFDH, FFDH) are faster but produce strictly larger gaps than BLF on these benchmarks; they did not win any instance.

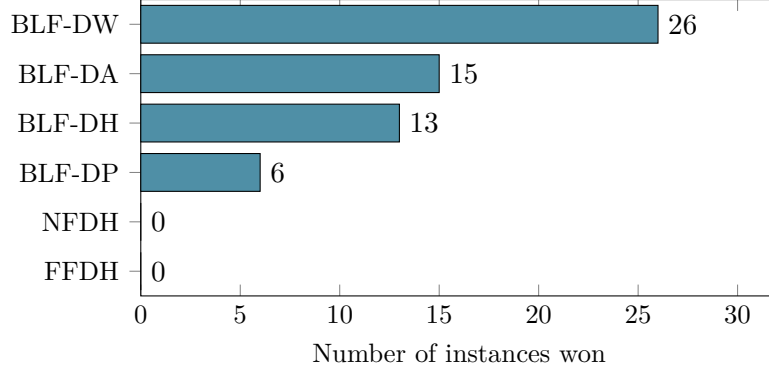


Figure 4: Proportion of instances won by each heuristic (training labels for the ML model).

4. CP-SAT finds optimal or improved solutions for small instances ($n \leq 20$) but is impractical beyond $n = 25$ within a 60 s budget.
5. The Random Forest ML selector correctly predicts the best heuristic and can reduce unnecessary human tuning overhead.

Future work includes adding metaheuristic approaches (simulated annealing, genetic algorithms), supporting rectangle rotation, and expanding the training set with synthetic instances to improve ML generalisation.

References

- [1] B. S. Baker, E. G. Coffman Jr., and R. L. Rivest, “Orthogonal packings in two dimensions,” *SIAM Journal on Computing*, vol. 9, no. 4, pp. 846–855, 1980.
- [2] B.-E. Bengtsson, “Packing rectangular pieces — a heuristic approach,” *The Computer Journal*, vol. 25, no. 3, pp. 353–357, 1982.
- [3] J. O. Berkey and P. Y. Wang, “Two-dimensional finite bin-packing algorithms,” *Journal of the Operational Research Society*, vol. 38, pp. 423–429, 1987.
- [4] B. Chazelle, R. L. Drysdale, and D. T. Lee, “Computing the largest empty rectangle,” *SIAM Journal on Computing*, vol. 15, no. 1, pp. 300–315, 1986.
- [5] E. G. Coffman Jr., M. R. Garey, D. S. Johnson, and R. E. Tarjan, “Performance bounds for level-oriented two-dimensional packing algorithms,” *SIAM Journal on Computing*, vol. 9, no. 4, pp. 808–826, 1980.
- [6] M. R. Garey and D. S. Johnson, *Computers and Intractability: A Guide to the Theory of NP-Completeness*. Freeman, 1979.
- [7] S. Martello and D. Vigo, “Exact solution of the two-dimensional finite bin packing problem,” *Management Science*, vol. 44, no. 3, pp. 388–399, 1998.
- [8] L. Perron and V. Furnon, *OR-Tools*, Google LLC, 2023. Available: <https://developers.google.com/optimization>